

Virtual Event Scheduler and Management Platform

Spring 2025 – 26 | June 2026

20210601045	Öykü İrem Oplazgil
-------------	--------------------

20230601014	Batuhan Bulut
-------------	---------------

20210601047	Eray Özkan
-------------	------------

20210601059	Mehmet Taşoğlu
-------------	----------------

Table of Contents

1. Introduction	3
2. Requirements	4
2.1 Functional Requirements	4
2.2 Non-Functional Requirements	5
3. Database Schema	6
3.1 Overview	6
3.2 Entity Relationships	6
3.3 – 3.5 Table Definitions	7
3.6 SQL Creation Scripts	8
4. LINQ Queries	9
5. Delegates and Events	12
5.1 Overview	12
5.2 Delegate Type Definitions	12
5.3 NotificationService	13
5.4 Usage in Controllers	14
5.5 Summary Table	14
6. MVC Structure	15
7. Sources	18
8. User Interface Design	19
9. Tests	21
9.1 Overview	21
9.2 Test Results	21
9.3 Test Case Details	22
9.4 Test Code Samples	23
10. Conclusion	25

1. Introduction

The **Virtual Event Scheduler and Management Platform** is a full-stack application developed as part of the SE 410 Software Framework Applications course. The platform enables organizations to create, manage, and promote virtual events while allowing attendees to discover events, register, and receive reminders.

The system is built using modern .NET technologies and follows a layered architecture consisting of a shared REST API, a web application for attendees, and desktop applications for staff and administrators. A central SQLite database ensures data consistency across all clients.

Project Objectives

- Provide an intuitive platform for scheduling and managing virtual events
- Enable attendees to browse events, register, and track their schedules
- Give administrators and staff tools to manage events, view participants, and control capacity
- Demonstrate key C# concepts including delegates, events, LINQ, and MVC architecture

Technology Stack

Component	Technology
Backend API	ASP.NET Core 10.0 Web API
Web Application	ASP.NET Core MVC + Razor Views
Desktop (Windows)	Windows Forms (.NET 10.0)
Desktop (macOS)	.NET MAUI (net9.0-maccatalyst)
Database	SQLite via Entity Framework Core 10.0
Authentication	JWT Bearer Tokens + BCrypt
Frontend Styling	Bootstrap 5 + Custom CSS
Testing	xUnit + Moq + EF Core InMemory

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR-01	Users shall be able to register with full name, email, and password
FR-02	Users shall be able to log in and receive a JWT authentication token
FR-03	Admins shall be able to create new virtual events with title, description, date/time, location, and capacity
FR-04	Admins and Staff shall be able to edit existing event details
FR-05	Admins and Staff shall be able to cancel events
FR-06	Authenticated users shall be able to register for active events that are not full
FR-07	Users shall be able to view all events they are registered for
FR-08	Users shall be able to unregister from events
FR-09	Admins and Staff shall be able to view the full participant list for any event
FR-10	The system shall filter events by date range and status using LINQ
FR-11	Admins shall be able to change user roles (Admin / Staff / Attendee)
FR-12	The system shall notify relevant parties when events are created, updated, cancelled, or registrations occur

2.2 Non-Functional Requirements

ID	Requirement
NFR-01	The application shall use JWT tokens for stateless authentication
NFR-02	Passwords shall be hashed using BCrypt before storage
NFR-03	The system shall run on both Windows (WinForms) and macOS (MAUI) for the desktop client
NFR-04	The SQLite database path shall be relative to the executable, enabling deployment on any machine
NFR-05	API response times shall be under 500ms for standard CRUD operations
NFR-06	The codebase shall follow MVC architectural patterns
NFR-07	All major classes and methods shall include XML documentation comments

ID	Requirement
NFR-08	The web interface shall be responsive and work on modern browsers

3. Database Schema

3.1 Overview

The system uses a **SQLite database** managed by Entity Framework Core with Code-First migrations. The database consists of three tables: **Users**, **Events**, and **EventRegistrations**.

3.2 Entity Relationships

The database implements a many-to-many relationship between Users and Events through the EventRegistrations join table. A unique composite index on (EventId, UserId) prevents duplicate registrations.

- User (1) —< EventRegistration (N) >— (1) Event
- A User can register for many Events through EventRegistrations
- An Event can have many registered Users through EventRegistrations
- Cascade delete: removing an Event or User automatically removes their registrations

3.3 Users Table

Column	Type	Constraints
Id	INTEGER	PRIMARY KEY, AUTOINCREMENT
FullName	NVARCHAR(100)	NOT NULL
Email	NVARCHAR(255)	NOT NULL, UNIQUE
PasswordHash	TEXT	NOT NULL
Role	NVARCHAR(20)	NOT NULL, DEFAULT 'Attendee'
CreatedAt	DATETIME	NOT NULL

3.4 Events Table

Column	Type	Constraints
Id	INTEGER	PRIMARY KEY, AUTOINCREMENT
Title	NVARCHAR(200)	NOT NULL
Description	TEXT	NOT NULL
DateTime	DATETIME	NOT NULL

Column	Type	Constraints
Location	NVARCHAR(200)	NOT NULL
Capacity	INTEGER	NOT NULL
CreatedBy	INTEGER	NOT NULL, FOREIGN KEY → Users(Id)
Status	NVARCHAR(50)	NOT NULL, DEFAULT 'Active'
CreatedAt	DATETIME	NOT NULL

3.5 EventRegistrations Table

Column	Type	Constraints
Id	INTEGER	PRIMARY KEY, AUTOINCREMENT
EventId	INTEGER	NOT NULL, FK → Events(Id) ON DELETE CASCADE
UserId	INTEGER	NOT NULL, FK → Users(Id) ON DELETE CASCADE
RegisteredAt	DATETIME	NOT NULL
—	UNIQUE	(EventId, UserId)

3.6 SQL Creation Scripts

```
CREATE TABLE Users (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    FullName NVARCHAR(100) NOT NULL,  
    Email NVARCHAR(255) NOT NULL UNIQUE,  
    PasswordHash TEXT NOT NULL,  
    Role NVARCHAR(20) NOT NULL DEFAULT 'Attendee',  
    CreatedAt DATETIME NOT NULL  
);  
  
CREATE TABLE Events (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    Title NVARCHAR(200) NOT NULL,  
    Description TEXT NOT NULL,  
    DateTime DATETIME NOT NULL,  
    Location NVARCHAR(200) NOT NULL,
```

```
Capacity INTEGER NOT NULL,  
CreatedBy INTEGER NOT NULL REFERENCES Users(Id),  
Status NVARCHAR(50) NOT NULL DEFAULT 'Active',  
CreatedAt DATETIME NOT NULL  
);  
  
CREATE TABLE EventRegistrations (  
    Id INTEGER PRIMARY KEY AUTOINCREMENT,  
    EventId INTEGER NOT NULL REFERENCES Events(Id) ON DELETE CASCADE,  
    UserId INTEGER NOT NULL REFERENCES Users(Id) ON DELETE CASCADE,  
    RegisteredAt DATETIME NOT NULL,  
    UNIQUE(EventId, UserId)  
);
```


4. LINQ Queries

LINQ (Language Integrated Query) is used throughout the application for all database operations. Queries are type-safe, composable, and translated to optimized SQL by Entity Framework Core at runtime.

4.1 Filter Events by Date Range and Status

Used in **EventsController.GetEvents()**. Builds a dynamic IQueryable query — the database receives a single optimized SQL statement regardless of which filters are active.

```
var query = _context.Events
    .Include(e => e.Registrations)
    .AsQueryable();

if (startDate.HasValue)
    query = query.Where(e => e.DateTime >= startDate.Value);

if (endDate.HasValue)
    query = query.Where(e => e.DateTime <= endDate.Value);

if (!string.IsNullOrEmpty(status))
    query = query.Where(e => e.Status == status);

var events = await query
    .OrderBy(e => e.DateTime)
    .Select(e => new EventDto { Id = e.Id, Title = e.Title, ... })
    .ToListAsync();
```

4.2 Get Events Registered by a Specific User

Used in **EventsController.GetMyEvents()**. Joins EventRegistrations with Events using navigation properties, retrieves all events the logged-in user is registered for, ordered by date.

```
var myEvents = await _context.EventRegistrations
    .Where(er => er.UserId == userId)
    .Include(er => er.Event)
        .ThenInclude(e => e.Registrations)
    .Select(er => new EventDto { ... })
    .OrderBy(e => e.DateTime)
    .ToListAsync();
```

4.3 Check for Duplicate Registration

Used in **RegistrationsController.RegisterForEvent()**. AnyAsync with a compound predicate efficiently checks for an existing registration before insertion.

```
bool alreadyRegistered = await _context.EventRegistrations
    .AnyAsync(er => er.EventId == eventId && er.UserId == userId);
```

4.4 Get Participants for an Event (Sorted)

Used in **RegistrationsController.GetEventParticipants()**. Retrieves participants with user data via navigation properties, ordered by registration date.

```
var participants = await _context.EventRegistrations
    .Where(er => er.EventId == eventId)
    .Include(er => er.User)
    .OrderBy(er => er.RegisteredAt)
    .Select(er => new {
        er.User.Id, er.User.FullName,
        er.User.Email, er.RegisteredAt
    })
    .ToListAsync();
```

4.5 Get All Users (Admin Panel)

Used in **AuthController.GetUsers()**. Orders by ID and projects into UserDto objects using LINQ Select.

```
var users = await _context.Users
    .OrderBy(u => u.Id)
    .Select(u => new UserDto {
        Id = u.Id,  FullName = u.FullName,
        Email = u.Email,  Role = u.Role,
        CreatedAt = u.CreatedAt
    })
    .ToListAsync();
```

5. Delegates and Events

5.1 Overview

The **NotificationService** class (VirtualEventScheduler.API/Services/NotificationService.cs) implements a delegate-based notification system. Four custom delegate types are defined, each corresponding to a specific platform event. The C# event keyword exposes delegates safely — external classes can only subscribe (+) or unsubscribe (-), not invoke them directly.

5.2 Custom Delegate Type Definitions

Each delegate type carries exactly the data needed for its notification scenario:

```
// Fired when a new event is created
public delegate void EventCreatedDelegate(
    string eventTitle, DateTime eventDate, string createdByName);

// Fired when a user successfully registers for an event
public delegate void UserRegisteredDelegate(
    string userFullName, string userEmail,
    string eventTitle, DateTime eventDate);

// Fired when an event is cancelled
public delegate void EventCancelledDelegate(
    string eventTitle, string cancelledByName);

// Fired when event details are updated
public delegate void EventUpdatedDelegate(
    string eventTitle, string updatedByName);
```

5.3 NotificationService Class

Registered as a **Singleton** in the DI container so that all delegate subscriptions persist for the lifetime of the application. Default logging handlers are subscribed in the constructor:

```
public class NotificationService
{
    private readonly ILogger<NotificationService> _logger;

    public event EventCreatedDelegate? OnEventCreated;
    public event UserRegisteredDelegate? OnUserRegistered;
```

```

public event EventCancelledDelegate? OnEventCancelled;
public event EventUpdatedDelegate? OnEventUpdated;

public NotificationService(ILogger<NotificationService> logger)
{
    _logger = logger;
    OnEventCreated += LogEventCreated;
    OnUserRegistered += LogUserRegistered;
    OnEventCancelled += LogEventCancelled;
    OnEventUpdated += LogEventUpdated;
}

public void NotifyEventCreated(string title, DateTime date, string by)
    => OnEventCreated?.Invoke(title, date, by);

public void NotifyUserRegistered(string name, string email, string title, DateTime date
)
    => OnUserRegistered?.Invoke(name, email, title, date);

public void NotifyEventCancelled(string title, string by)
    => OnEventCancelled?.Invoke(title, by);

public void NotifyEventUpdated(string title, string by)
    => OnEventUpdated?.Invoke(title, by);

private void LogEventCreated(string title, DateTime date, string by)
    => _logger.LogInformation(
        "[NOTIFICATION] Event created: '{Title}' on {Date} by {User}",
        title, date, by);

// ... (similar handlers for Registered, Cancelled, Updated)
}

```

5.4 Usage in Controllers

After each successful database save, the appropriate Notify method is called:

```

// EventsController.CreateEvent() – after SaveChangesAsync()
_notificationService.NotifyEventCreated(
    newEvent.Title, newEvent.DateTime, creator?.FullName ?? "Unknown");

// EventsController.CancelEvent()
_notificationService.NotifyEventCancelled(
    eventItem.Title, canceller?.FullName ?? "Unknown");

```

```
// RegistrationsController.RegisterForEvent()  
_notificationService.NotifyUserRegistered(  
    user.FullName, user.Email, eventItem.Title, eventItem.DateTime);
```

5.5 Delegate Functionality Summary

Delegate	Trigger	Parameters
EventCreatedDelegate	New event created by Admin/Staff	eventTitle, eventDate, createdByName
UserRegisteredDelegate	Attendee registers for an event	userFullName, userEmail, eventTitle, eventDate
EventCancelledDelegate	Event cancelled by Admin/Staff	eventTitle, cancelledByName
EventUpdatedDelegate	Event details edited by Admin/Staff	eventTitle, updatedByName

6. MVC Structure

6.1 Overview

The project consistently applies the **Model-View-Controller** architectural pattern across both the REST API and the web application layers, providing clean separation of concerns between data representation, presentation, and business logic.

6.2 Models

Models are defined in the **VirtualEventScheduler.Data** shared project and reused across all layers:

Model / DTO	Location	Purpose
User.cs	Data/Models	Platform user — Admin, Staff, or Attendee role
Event.cs	Data/Models	Virtual event with capacity, status, and scheduling
EventRegistration.cs	Data/Models	Join entity linking Users to Events
EventDto.cs	API/DTOs	API response shape for event data
EventCreateDto.cs	API/DTOs	Input shape for creating events
EventUpdateDto.cs	API/DTOs	Input shape for updating events
LoginDto / RegisterDto	API/DTOs	Authentication request bodies
EventViewModel.cs	Web/Models	View model for event pages
ParticipantViewModel.cs	Web/Models	View model for participant list

6.3 Views

Web views are implemented as **Razor (.cshtml)** pages. The shared layout provides consistent navigation, branding, and flash messages across all pages:

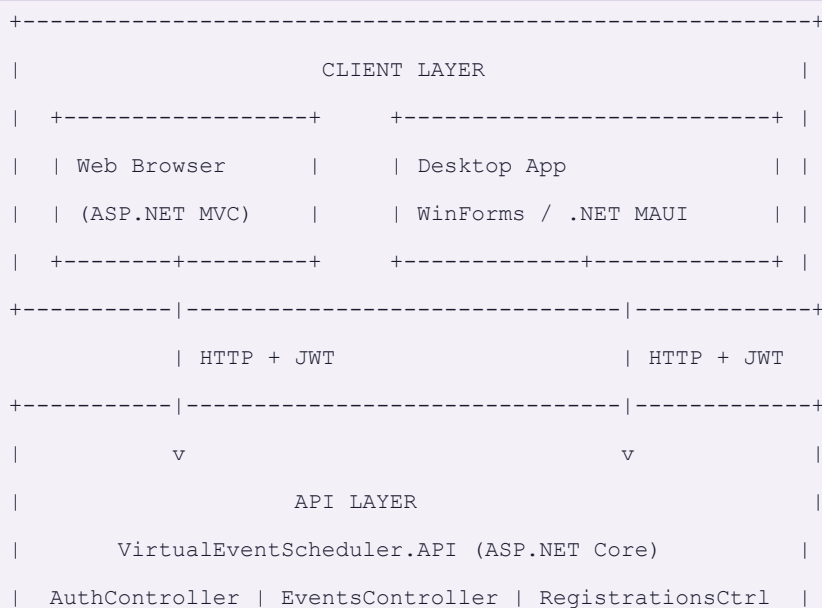
View	Path	Description
Index.cshtml	Views/Events/	Event listing with filter and capacity progress bars
Details.cshtml	Views/Events/	Full event info, registration, admin controls
Create.cshtml	Views/Events/	Event creation form (Admin/Staff only)
Edit.cshtml	Views/Events/	Event editing form (Admin/Staff only)

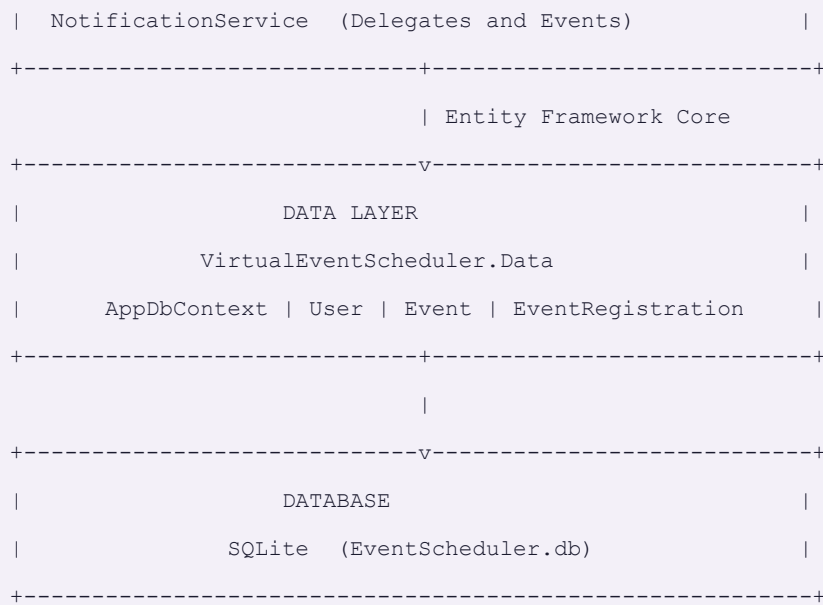
View	Path	Description
MyEvents.cshtml	Views/Events/	User's registered events with reminders
Participants.cshtml	Views/Events/	Participant list (Admin/Staff only)
Login.cshtml	Views/Account/	Login card form
Register.cshtml	Views/Account/	Registration card form
_Layout.cshtml	Views/Shared/	Master layout: navbar, footer, flash messages

6.4 Controllers

Controller	Layer	Responsibilities
AuthController	API	Register, login, list users, update roles
EventsController	API	CRUD for events, cancel, my events endpoint
RegistrationsController	API	Register/unregister for events, list participants
AccountController	Web	Web login, register, logout, user management
EventsController	Web	All event pages: index, details, create, edit, cancel, participants, my events
HomeController	Web	Landing page with hero section

6.5 Architecture Diagram





6.6 Component Selection Rationale

- ASP.NET Core MVC — built-in session management, Razor view engine, tight .NET integration
- Windows Forms — rapid desktop UI development with mature data-binding support
- .NET MAUI — recommended cross-platform UI framework for .NET desktop apps on macOS
- Entity Framework Core + SQLite — zero-configuration, file-based database ideal for portable deployment
- JWT Bearer authentication — stateless API security compatible with both browser sessions and desktop HTTP clients

7. Sources

7.1 GitHub Repository

The complete source code for both Version 1 and Version 2 is publicly available at:

<https://github.com/serhatuzunbayir/VolunteerMatchingSPRING2025-26>

7.2 How to Run the Project

Prerequisites

- .NET 9 or .NET 10 SDK — <https://dotnet.microsoft.com/download>
- macOS users additionally need: dotnet workload install maui

Step 1 — Clone the repository

```
git clone https://github.com/serhatuzunbayir/VolunteerMatchingSPRING2025-26.git
cd VolunteerMatchingSPRING2025-26/VirtualEventScheduler
```

Step 2 — Start the API (Terminal 1)

```
dotnet run --project VirtualEventScheduler.API/VirtualEventScheduler.API.csproj
# API:      http://localhost:5202
# Swagger:  http://localhost:5202/swagger
```

Step 3 — Start the Web Application (Terminal 2)

```
dotnet run --project VirtualEventScheduler.Web/VirtualEventScheduler.Web.csproj
# Open browser at: http://localhost:5050
```

Step 4 — Start the Desktop Application (Terminal 3)

```
# Windows:
dotnet run --project VirtualEventScheduler.Desktop/VirtualEventScheduler.Desktop.csproj

# macOS:
dotnet run --project VirtualEventScheduler.Desktop.Maui/VirtualEventScheduler.Desktop.Maui.csproj -f net9.0-maccatalyst
```

Default Credentials

Role	Email	Password
Admin	admin@eventapp.com	Admin123!

Role	Email	Password
Demo Attendee	mehmet@eventapp.com	Demo1234!
Demo Attendee	oyku@eventapp.com	Demo1234!
Demo Attendee	batuhan@eventapp.com	Demo1234!

7.3 Video Demonstrations

Version	Location
Version 1 Demo	Submitted via Blackboard — SE 410 Spring 2025-26 Course Submission Portal
Version 2 Demo	Submitted via Blackboard — SE 410 Spring 2025-26 Course Submission Portal

8. User Interface Design

8.1 Web Application

The web application is built with **Bootstrap 5** and a custom CSS design system centred on a deep purple brand identity (#5C2D91).

Navigation Bar

A deep purple gradient navbar (from #2d1554 to #7b3fbf) displays the application brand name, navigation links, and the current user's name with a colour-coded role badge: red for Admin, yellow for Staff, and grey for Attendee. Navigation links adapt dynamically based on login state and role — My Events and Create Event only appear for relevant roles.

Events Listing Page

Displays all events as a responsive 3-column card grid. Each card shows: event title, description excerpt, date/time and location with emoji icons, a dynamic **capacity progress bar** (green below 60%, yellow 60–90%, red above 90%), a status badge (Active/Completed/Cancelled), and a **Full** warning badge when at capacity. A filter panel enables filtering by date range and status.

Event Details Page

Shows full event information including all fields. Authenticated users see a Register button on active non-full events. Admin and Staff users additionally see **Edit Event**, **Cancel Event** (with confirmation dialog), and **View Participants** buttons.

My Events Page

Lists all events the logged-in user is registered for. Events within the next 7 days receive a gold left-border highlight and a reminder badge showing *Today!*, *Tomorrow!*, or *In X days*.

Create / Edit Event Forms

Card-style forms with a purple gradient card header, labelled form controls, and client-side validation feedback rendered inline beneath each field.

Authentication Pages

Login and Register pages are centred card layouts with a purple top-accent border, descriptive placeholder text, and full-width primary action buttons.

8.2 Desktop Application (Windows Forms)

The Windows Forms application targets **Admin and Staff** users, providing event management tools. It communicates with the shared REST API via HTTP.

Login Form

Email and password fields with a login button. Only Admin and Staff roles may access the desktop application.

Main Form

Displays a welcome message with the user's name and role, and a navigation button opening the Event Management form.

Event Management Form

The primary screen contains: an event-creation panel (title, description, date/time, location, capacity), a **DataGridView** listing all events with columns for ID, title, date, location, registered count, capacity, and status, a status filter dropdown, a **View Participants** button opening the participant dialog for the selected row, and a **Cancel Event** button with a confirmation dialog.

Participants Form

A dialog showing a DataGridView of all participants registered for the selected event, with name, email address, and registration timestamp.

Note: Full GUI screenshots of both the web and desktop applications are demonstrated live during the project presentation.

9. Tests

9.1 Overview

The project includes a dedicated test suite in **VirtualEventScheduler.Tests/** using:

- **xUnit** — test framework
- **Moq** — mocking library for ILogger dependencies
- **Microsoft.EntityFrameworkCore.InMemory** — isolated in-memory database for controller tests

Run all tests:

```
dotnet test VirtualEventScheduler.Tests/VirtualEventScheduler.Tests.csproj
```

9.2 Test Results

Metric	Result
Total Tests	10
Passed	10 
Failed	0
Skipped	0
Duration	~604 ms

9.3 Test Case Details

NotificationServiceTests (5 tests)

#	Test Name	Description	Result
1	NotifyEventCreated_Should FireOnEventCreatedDelegat e	Verifies OnEventCreated delegate fires when NotifyEventCreated() is called. A boolean flag is set by the handler and asserted true.	PASSED
2	NotifyEventCreated_Should PassCorrectTitleToDelegat e	Verifies the correct event title string is passed to the delegate handler.	PASSED

#	Test Name	Description	Result
3	NotifyUserRegistered_Should FireAndCaptureUserEmail	Verifies OnUserRegistered fires and correctly passes the user's email to the handler.	PASSED
4	NotifyEventCancelled_Should FireOnEventCancelledDelegate	Verifies OnEventCancelled fires and propagates the event title to the handler.	PASSED
5	NotifyEventUpdated_Should FireOnEventUpdatedDelegate	Verifies OnEventUpdated fires when NotifyEventUpdated() is called.	PASSED

EventsControllerTests (5 tests)

#	Test Name	Description	Result
6	GetEvents_WhenNoEventsExist_ReturnsEmptyList	Empty in-memory DB. GetEvents() should return HTTP 200 with an empty collection.	PASSED
7	GetEvents_WithStatusFilter_ReturnsOnlyMatching	Seeds Active and Cancelled events. Filtering by 'Active' asserts exactly one result — validating the LINQ Where clause.	PASSED
8	CreateEvent_Should PersistEventToDatabase	Calls CreateEvent() as Admin with valid DTO. Asserts event count in DB is 1 and response is HTTP 201.	PASSED
9	CancelEvent_Should ChangeStatusToCancelled	Seeds Active event, calls CancelEvent(). Asserts Status field is 'Cancelled' in the database.	PASSED
10	GetEvent_WithInvalidId_ReturnsNotFound	Calls GetEvent(999) on empty database. Asserts HTTP 404 NotFound is returned.	PASSED

9.4 Test Code Samples

Delegate Test Example

```
[Fact]
public void NotifyEventCreated_ShouldFireOnEventCreatedDelegate()
{
    // Arrange
    var logger = new Mock<ILogger<NotificationService>>();
    var service = new NotificationService(logger.Object);
    bool wasFired = false;
    service.OnEventCreated += (title, date, createdBy) => wasFired = true;
```

```
// Act
service.NotifyEventCreated("Tech Talk 2026",
                           DateTime.Now.AddDays(7), "Admin User");

// Assert
Assert.True(wasFired, "OnEventCreated delegate should have fired.");
}
```

LINQ Filter Test Example

```
[Fact]
public async Task GetEvents_WithStatusFilter_ReturnsOnlyMatchingEvents()
{
    // Arrange – seed one Active and one Cancelled event
    using var db = CreateDb(nameof(GetEvents_WithStatusFilter));
    db.Events.Add(new Event {
        Title = "Active Event",    Status = "Active",
        DateTime = DateTime.Now.AddDays(1), Capacity = 10,
        Location = "Online", CreatedBy = 1, Description = "desc"
    });
    db.Events.Add(new Event {
        Title = "Cancelled Event", Status = "Cancelled",
        DateTime = DateTime.Now.AddDays(2), Capacity = 10,
        Location = "Online", CreatedBy = 1, Description = "desc"
    });
    await db.SaveChangesAsync();
    var controller = CreateController(db);

    // Act – LINQ filter by status = "Active"
    var result = await controller.GetEvents(null, null, "Active");

    // Assert – only the Active event returned
    var ok = Assert.IsType<OkObjectResult>(result.Result);
    var events = Assert.IsAssignableFrom<IEnumerable<EventDto>>(ok.Value).ToList();
    Assert.Single(events);
    Assert.Equal("Active Event", events[0].Title);
}
```

10. Conclusion

The **Virtual Event Scheduler and Management Platform** successfully demonstrates the integration of modern .NET technologies into a cohesive, cross-platform full-stack application. Version 2 extends the original foundation with a complete ASP.NET Core MVC web application, a delegate-based notification system, full event lifecycle management (create, edit, cancel), and participant tracking across web and desktop clients.

Key technical achievements include the implementation of custom C# **delegates and events** through the NotificationService, providing a decoupled, extensible notification mechanism that automatically fires on all significant system events. **LINQ** is used extensively throughout the data access layer for dynamic filtering, sorting, joining, and projecting data efficiently — with all queries translated to optimized SQL by Entity Framework Core.

The **MVC architectural pattern** is consistently applied across both the web and API layers, cleanly separating data models, presentation views, and business logic controllers. The application is fully **cross-platform**: the API and web application run identically on Windows and macOS, while dedicated desktop clients target Windows (Windows Forms) and macOS (.NET MAUI). The SQLite database uses a path relative to the executable directory, enabling zero-configuration deployment on any machine.

With a complete unit test suite of **10 passing tests** covering delegate behaviour, LINQ filtering, event creation, cancellation, and error handling, the project demonstrates not only functional software but a commitment to code quality and testability expected of professional-grade applications.